Assignment : T1

Q1. Illustrate resource sharing in a distributed system with a web-based example, listing transparency requirements.

Ans. web-based example: online Food delivery Platform (eg. Swiggy/Uber Eats).
The food delivery website is not one programm or one server. It's made up of many distributed services.

-> Restaurant service
-> order service
-> payment service
-> Delievery tracking service
-> user profile service

All of these share resources such as:

-> Databases
-> APIs
-> servers
-> network bandwidth

Resource sharing explained step-by-step

1. user Action:
A customer opens the website and places an order.

what actually happens:

1. The order service access a shared database to create an order.
2. The payment service access a shared payment gateway

3 The Delievery service accesses shared location servers

These resources are shared concurrently across many machines.

Transparency requirements

1. Access Transparency:
users uses the same web interface regardless of which service handles the request.
2. location Transparency:
user don't know where service or databases are hosted.
3. Concurrency Transparency:
Thousands of user place order at the same time without interference.
4. Replication Transparency:
Databases and services are replicated to improve speed and reliability.
5. Failure Transparency
If one service instance crashes, another handles the payment
6. performance Transparency:
The system adapts under heavy load.

Q2. Compare client-server and peer-to-peer architectural models with diagrams.

Ans. 1. Client-Server Architecture

How it works

-> many users want the same service
-> one powerful machine owns the data and logic
-> Everyone else must ask permission to use it

Work Flow

1. Client sends request
2. Server processes it
3. Server send response
4. Client waits and depends on server

Diagram

[client] ----> request ----> [server]
[client] <---- response ---- [server]
[client] ----> request ----> [server]

Peer-to-peer Architecture

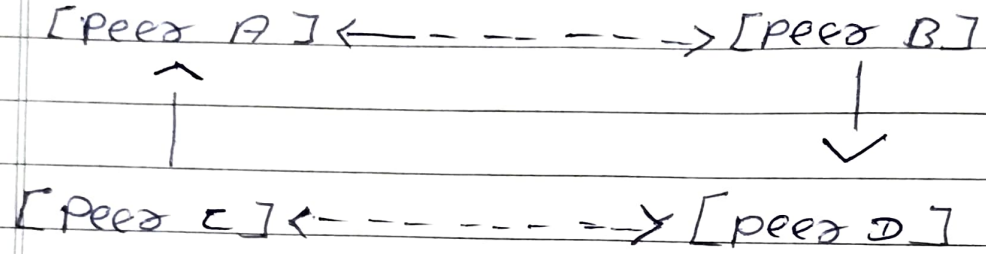
How it works

-> Every system has something to offer
-> No single owner of data or control
-> Nodes cooperate and share directly

work flow

1. Peer requests data from another peer
2. peer responds
3. Roles change dynamically

Diagram



Q3 Implement Lamport's logical clock for three processes exchanging messages (Provide timestamp sequence).

Ans. Lamport's logical clock – Rules (quick recap)

Each process P_i maintains a logical clock c_i.

Rules:

1. Before every local event:
   $c_i = c_i + 1$
2. when sending a message m:
   → increment clock
   → Attach timestamp c_i to m
3. when receiving a message with timestamp T:
   $c_i = \max(c_i, T) + 1$

Three processes

Let

→ P1 with clock C1

→ P2 with clock C2

→ P3 with clock C3

step-by-step Execution with Timestamp

Event-1 : P1 sends message m1 to P2

P1 : C1 = 2

m1 carries timestamp 2

step2 P2 receives m1

$C_2 = max(0,2) + 1 = 3$

step 3 concurrent local event at P3

P3 : C3 = 1

step 4 P2 sends messages m2 to P3

P2 : C2 = 4

m2 carries timestamp 4

Step5: P3 receives m2

$C3 = max(1, 4) + 1 = 5$

step6: P3 sends message m3 to P1

P3 : C3 = 6

m3 carries timestamp 6

step 7: P1 receives m3
C1 = max(2, 6) + 1 = 7

Final logical clock values
P1 = 7 P2 = 4 P3 = 6

Q4. Analyze limitations of distributed system due to absence of global clock using a synchronization scenario.

Ans. Core problem: No Global Clock
→ Each process has its own local clock
→ clock drift and cannot be perfectly synchronized
→ message delays are unpredictable

Synchronization scenario:
Goal
Ensures only one process enters a critical section at a time.

Problem caused by absence of global clock
Event sequence (Real time)

Real time:
P1 sends message at t = 10
P2 sends message at t = 11

local clock readings:
P1 clock = 100
P2 clock = 80

observed Timestamp
P1 request → timestamp 100
P2 request → timestamp 80

core limitations
1. No reliable notion of "now"
→ Local times cannot be compared safely
→ same timestamp ≠ same real time
2. waiting becomes uncertain:
A process cannot know:
→ If another process is slow
→ or has failed
→ or its message is delayed
3. False failure detection
If P1 doesn't hear from P2:
→ Is P2 crashed?
→ or just slow?
4. Extra communication needed
Instead of time:
→ processes exchange "I'm done" messages
→ often require acknowledgements

Q5. Design a simple token-based mutual exclusion algorithms For 4 nodes and simulate message Flow.

Ans. Token-Based mutual Exclusion (4 nodes, request-driven)

1. System Assumptions:
→ Nodes: P1, P2, P3, P4
→ Exactly one token exists in the system
→ Token can be sent directly to a requesting node
→ Reliable message delivery

2. Data structures
At each node Pi
→ wantCS : boolean
→ RN[1..4]: request numbers (initially 0)
Total structure
→ LN[1..4]: last satisfied request numbers
→ Q: queue of requesting nodes

3. Algorithms (simplified)
Request Critical section (at Pi)
RN[i] = RN[i] + 1
broadcast REQUEST(i, RN[i])
want CS = true

on receiving REQUEST(j, n)
RN[j] = max(RN[j], n)
IF hasToken and wantCS == False and RN[j] == LN[j]+1
send Token to PJ
hasToken = False

on receiving Token
has Token = true
enter CS

Exit CS
LN[i] = RN[i]
For each j:
IF RN[j] == LN[j] + 1;
add j to Q

If Q not empty:
send TOKEN to first process in Q
has Token = False

05/01